In [1]:
```
#Micro-Project 3 should use machine learning regression techniques including but no

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(
    r'C:\Users\jonat\OneDrive\Desktop\NU Homework\ANA500\ANA500Project\ANA500Projec
    parse_dates=['DateTime'],
    index_col='DateTime'
)

amp_cols = [c for c in df.columns if 'Marrakech' not in c]
kw_cols  = [c for c in df.columns if 'Marrakech' in c]

VOLTAGE = 220
POWER_FACTOR = 0.9

def amps_to_kw(current_amps, voltage=VOLTAGE, power_factor=POWER_FACTOR):
    return (voltage * current_amps * power_factor) / 1000

df_kw = df.copy()
df_kw[amp_cols] = df_kw[amp_cols].apply(amps_to_kw)

zone_cols = [c for c in df_kw.columns if c.startswith('zone')]

# Peak label per zone (90th percentile within that zone)
peak_labels = pd.DataFrame(index=df_kw.index)
for col in zone_cols:
    thresh = df_kw[col].quantile(0.90)
    peak_labels[f'peak_{col}'] = (df_kw[col] > thresh).astype(int)  # 1 = Peak, 0 =

df_kw = df_kw.join(peak_labels)
```

In [5]:
```
#Step Peak vs normal classification (checking what is in the 90th percentile per ci

city_map = {
    'Laayoune':    [c for c in df_kw.columns if 'Laayoune' in c],
    'Boujdour':    [c for c in df_kw.columns if 'Boujdour' in c],
    'FoumEloued':  [c for c in df_kw.columns if 'FoumEloued' in c],
    'Marrakech':   [c for c in df_kw.columns if 'Marrakech' in c],
}

# Total load per city = sum of that city's zone columns (in kW, post-conversion)
for city, cols in city_map.items():
    df_kw[f'total_{city}'] = df_kw[cols].sum(axis=1, skipna=True)

# Peak = load exceeds the 90th percentile of THAT city's own distribution
labels = pd.DataFrame(index=df_kw.index)
for city in city_map:
    threshold = df_kw[f'total_{city}'].quantile(0.90)
    labels[f'peak_{city}'] = (df_kw[f'total_{city}'] > threshold).map({True: 'Peak'
    print(f'{city}: 90th percentile threshold = {threshold:.2f} kW')
```

```
df_kw = df_kw.join(labels)
df_kw[[c for c in df_kw.columns if 'total_' in c or 'peak_' in c]].head(10)
```

```
Laayoune: 90th percentile threshold = 145.20 kW
Boujdour: 90th percentile threshold = 37.46 kW
FoumEloued: 90th percentile threshold = 129.85 kW
Marrakech: 90th percentile threshold = 1635.00 kW
```

Out[5]:

DateTime	total_Laayoune	total_Boujdour	total_FoumEloued	total_Marrakech	peak_Laay
2022-09-14 17:00:00	114.647820	26.803198	168.314692	0.0	Nc
2022-09-14 17:30:00	116.499817	26.879040	168.781371	0.0	Nc
2022-09-14 18:00:00	121.036208	27.850646	171.818346	0.0	Nc
2022-09-14 18:30:00	126.276927	27.491313	173.566312	0.0	Nc
2022-09-14 19:00:00	146.969838	27.003262	171.354033	0.0	
2022-09-14 19:30:00	149.960005	27.774984	173.514641	0.0	
2022-09-14 20:00:00	147.937502	32.322967	175.959425	0.0	
2022-09-14 20:30:00	147.212396	34.746930	177.301145	0.0	
2022-09-14 21:00:00	147.468064	35.391510	172.971298	0.0	
2022-09-14 21:30:00	147.208436	37.175091	172.054252	0.0	

In [11]:

```
# Reshape to long format (one row per timestamp × zone) and engineer features:

# Long format: each row is one (timestamp, zone) observation
long_rows = []
for col in zone_cols:
    temp = pd.DataFrame({
        'DateTime': df_kw.index,
        'kw': df_kw[col].values,
        'peak': df_kw[f'peak_{col}'].values,
        'zone_city': col
    })
    long_rows.append(temp)

long_df = pd.concat(long_rows, ignore_index=True).dropna(subset=['kw'])
```

```
long_df['city'] = long_df['zone_city'].str.extract(r'zone\d+_(\w+)')
long_df['zone'] = long_df['zone_city'].str.extract(r'(zone\d+)_')

# Time-based features
long_df['hour'] = long_df['DateTime'].dt.hour
long_df['dayofweek'] = long_df['DateTime'].dt.dayofweek
long_df['month'] = long_df['DateTime'].dt.month

long_df['zone_hour'] = long_df['zone_city'] + '_' + long_df['hour'].astype(str)

def month_to_season(month):
    if month in [12, 1, 2]: return 'Winter'
    elif month in [3, 4, 5]: return 'Spring'
    elif month in [6, 7, 8]: return 'Summer'
    else: return 'Fall'

long_df['season'] = long_df['month'].map(month_to_season)

long_df.head()
```

Out[11]:

	DateTime	kw	peak	zone_city	city	zone	hour	dayofweek	mont
0	2022-09-14 17:00:00	14.646063	0	zone1_Laayoune	Laayoune	zone1	17	2	
1	2022-09-14 17:30:00	15.678060	0	zone1_Laayoune	Laayoune	zone1	17	2	
2	2022-09-14 18:00:00	16.800154	0	zone1_Laayoune	Laayoune	zone1	18	2	
3	2022-09-14 18:30:00	18.339826	1	zone1_Laayoune	Laayoune	zone1	18	2	
4	2022-09-14 19:00:00	20.861580	1	zone1_Laayoune	Laayoune	zone1	19	2	

In [12]:

```
# Build the feature matrix (one-hot encode categoricals, scale numerics):

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatri

numeric_features = []  # none left as raw numeric
categorical_features = ['zone_city', 'season', 'dayofweek', 'hour', 'zone_hour']

X_sorted = long_df.sort_values('DateTime')[numeric_features + categorical_features]
y_sorted = long_df.sort_values('DateTime')['peak']

split_idx = int(len(X_sorted) * 0.8)
X_train, X_test = X_sorted.iloc[:split_idx], X_sorted.iloc[split_idx:]
y_train, y_test = y_sorted.iloc[:split_idx], y_sorted.iloc[split_idx:]
```

```
preprocessor = ColumnTransformer(transformers=[
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
])



# A chronological split (train on earlier dates, test on later ones) is more defens
```

In [13]:
```
# Fit both models

# --- Logistic Regression ---
logreg_pipeline = Pipeline([
    ('preprocess', preprocessor),
    ('clf', LogisticRegression(class_weight='balanced', max_iter=2000))
])
logreg_pipeline.fit(X_train, y_train)
y_pred_logreg = logreg_pipeline.predict(X_test)

svm_pipeline = Pipeline([
    ('preprocess', preprocessor),
    ('clf', LinearSVC(class_weight='balanced', max_iter=10000))
])
svm_pipeline.fit(X_train, y_train)
y_pred_svm = svm_pipeline.predict(X_test)

# class_weight='balanced' matters here — peak labels are ~90/10 imbalanced by const
```

In [14]:
```
# Evaluate both / metrics

print("=== Logistic Regression (with zone×hour interaction) ===")
print(classification_report(y_test, y_pred_logreg, target_names=['Normal', 'Peak']))
print("\n=== Linear SVM (with zone×hour interaction) ===")
print(classification_report(y_test, y_pred_svm, target_names=['Normal', 'Peak']))

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_logreg, display_labels=['Nor
axes[0].set_title('Logistic Regression')
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_svm, display_labels=['Normal
axes[1].set_title('Linear SVM')
plt.tight_layout()
plt.show()
```

```
=== Logistic Regression (with zone×hour interaction) ===
              precision    recall  f1-score   support

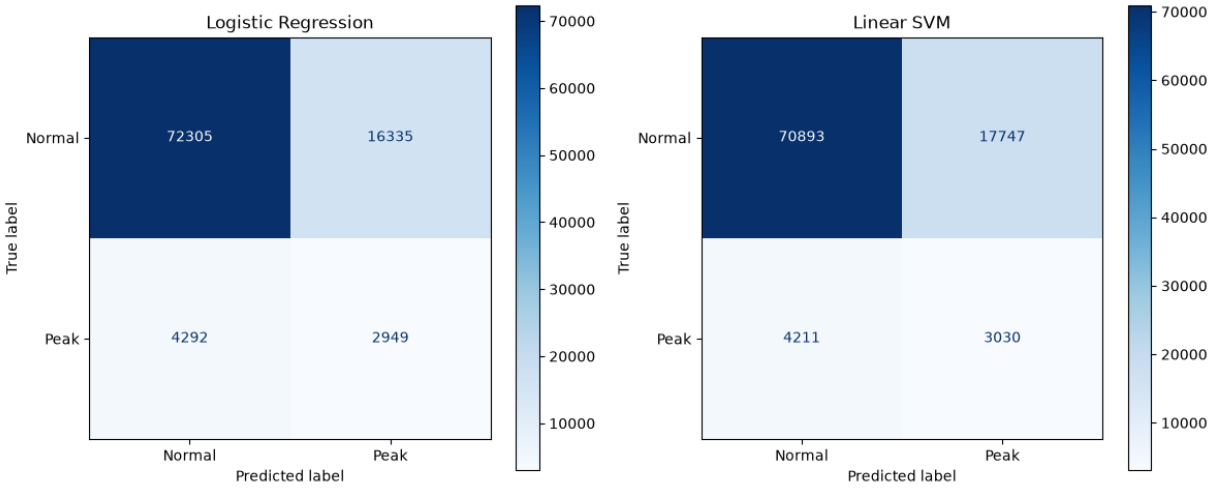
      Normal       0.94      0.82      0.88     88640
        Peak       0.15      0.41      0.22      7241

    accuracy                           0.78     95881
   macro avg       0.55      0.61      0.55     95881
weighted avg       0.88      0.78      0.83     95881


=== Linear SVM (with zone×hour interaction) ===
              precision    recall  f1-score   support

      Normal       0.94      0.80      0.87     88640
        Peak       0.15      0.42      0.22      7241

    accuracy                           0.77     95881
   macro avg       0.54      0.61      0.54     95881
weighted avg       0.88      0.77      0.82     95881
```



In [ ]: